

Дандер методы



ДМИТРИЙ СЕРГЕЕВ

Должность, компания

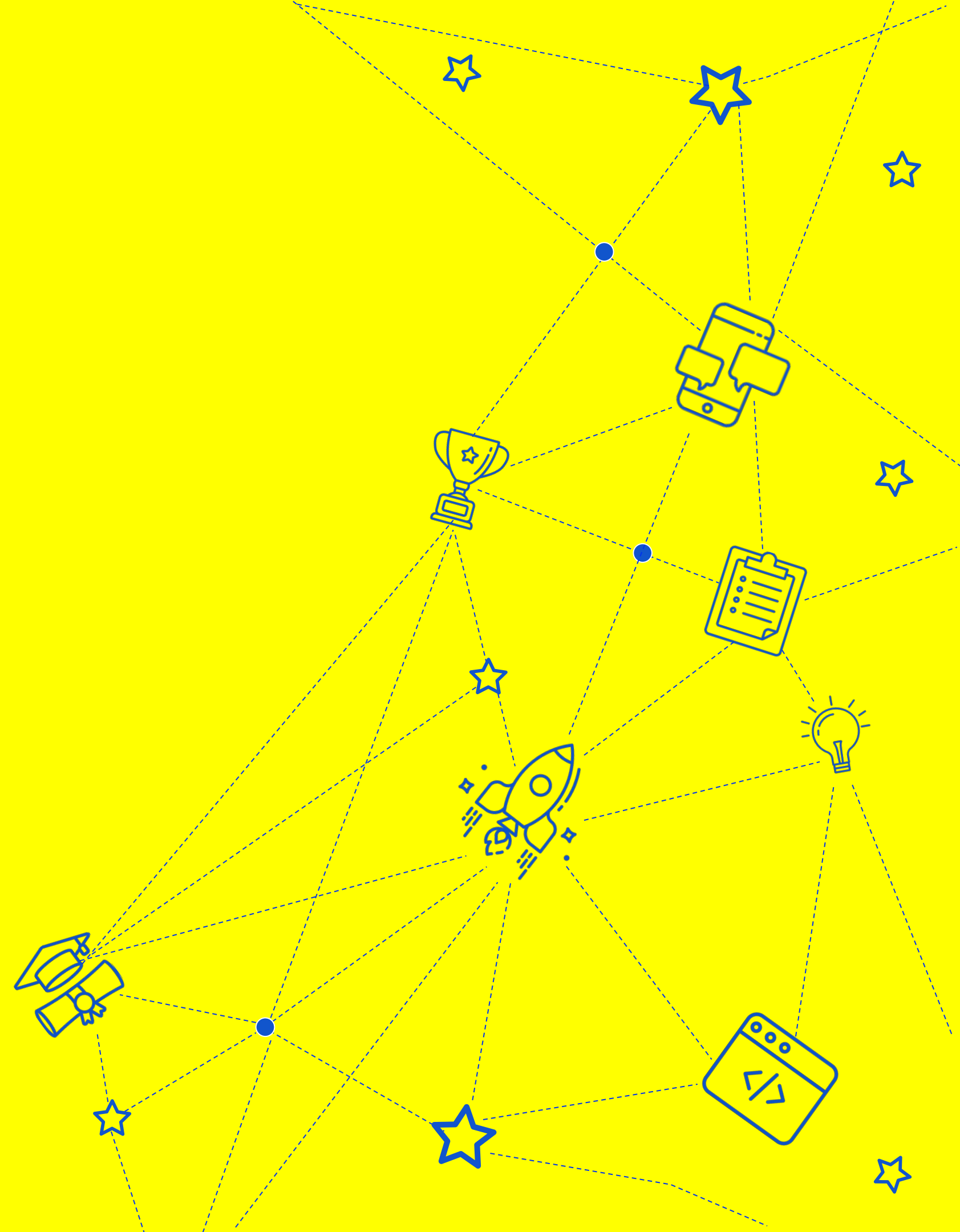
BAWAGA Seller - Руководитель
отдела backend разработки

- Самоучка, прошел путь от 0 до руководителя
- Вдохновляюсь исходным кодом
- Занимаюсь менторством около 2 лет

Содержание урока

- ★ Введение в dunder методы
- ★ Обзор dunder методов, часть 1
- ★ Обзор dunder методов, часть 2

Введение в dunder методы



Что такое dunder

Dunder methods («dunder» образованное от «double underscore») – неофициальное название методов, название которых начинается и заканчивается двойным нижним подчеркиванием `'__*__'` (например, `__init__`, `__str__` и т.д.).

Также dunder методы называют **«магическими» методами**.



В документации используется термин **Special methods**.

```
import sys
from types import ModuleType

class VerboseModule(ModuleType):
    def __repr__(self):
        return f'Verbose {self.__name__}'

    def __setattr__(self, attr, value):
        print(f'Setting {attr}...')
        super().__setattr__(attr, value)
```


В чем сила **магия**, брат?

```
1  class SomeClass:
2      def __init__(self, value) -> None:
3          print('Вызов метода __init__')
4          self._value = value
5
6      def __eq__(self, other):
7          print('Вызов метода __eq__')
8          return hash(self._value) == hash(other._value)
9
10     def equals(self, other):
11         print('Вызов метода equals')
12         return hash(self._value) == hash(other._value)
13
14  a = SomeClass(1)
15  # Вызов метода __init__
16  b = SomeClass(2)
17  # Вызов метода __init__
18  if a.equals(b):
19      print('is equals')
20  # Вызов метода equals
21
22
23  if a == b:
24      print('is equals')
25  # Вызов метода __eq__
```



Методы инициализации

`__new__(cls, [, ...])`

Статический метод, который вызывается для создания нового экземпляра класса. Используйте его, когда нужно контролировать создание нового экземпляра.

`__init__(self, [, ...])`

Метод вызывается после создания экземпляра.

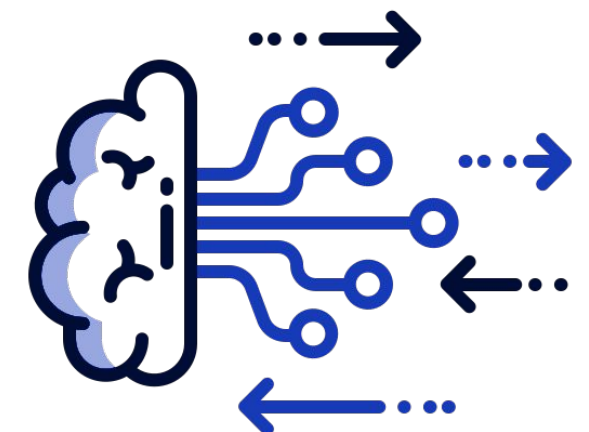
Используется часто или практически всегда. Используйте его, когда нужно управлять инициализацией нового экземпляра.

`__del__(self)`

Метод вызывается, когда объект собирается быть уничтоженным.

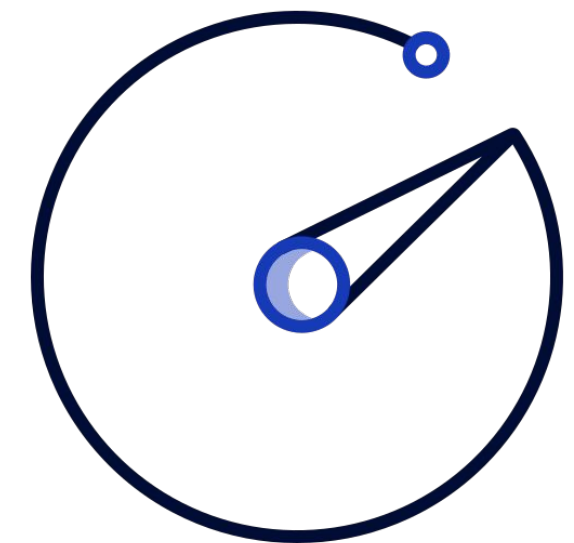
Методы инициализации

```
1  class String(str):
2      def __new__(cls, value):
3          obj = super().__new__(cls, value)
4          # to do something
5          return obj
6
7  class Thing:
8      def __init__(self, name):
9          self.name = name
10
11  class Book(Thing):
12      def __init__(self, author, *args, **kwargs):
13          super().__init__(*args, **kwargs)
14          self.author = author
15
16  don_quixote = Book(name='Don Quixote', author='Miguel de Cervantes')
```



Жизненный цикл объекта

```
1  class String(str):
2      def __new__(cls, value):
3          print('Вызов метода __new__')
4          return super().__new__(cls, value)
5
6      def __init__(self, value, *args, **kwargs):
7          print('Вызов метода __init__')
8          super().__init__(*args, **kwargs)
9
10     def action(self):
11         print('Вызов метода action')
12
13     def __del__(self):
14         print('Вызов метода __del__')
15
16 string = String('Hello world!')
17 # Вызов метода __new__
18 # Вызов метода __init__
19 string.action()
20 # Вызов метода action
21
22 # Выходим из программы
23 exit()
24
25 # Вызов метода __del__
```



Обзор dunder методов, часть 1



Методы сравнения

..... `__lt__(self, other)`

..... `__le__(self, other)`

..... `__eq__(self, other)`

..... `__ne__(self, other)`

..... `__gt__(self, other)`

..... `__ge__(self, other)`

..... `__hash__(self)`

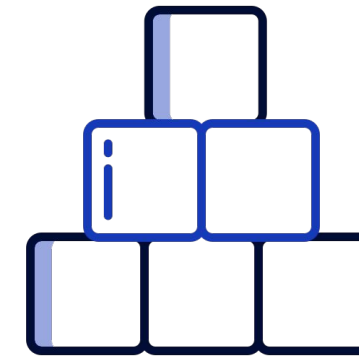
Возвращает результат сравнения операндов оператором `<`, `<=`, `=`, `!=`, `>` и `>=` соответственно.

Вызывается встроенной функцией `hash`.

Сравнение объектов `x == y` подразумевает, что `hash(x) == hash(y)`.

Методы сравнения

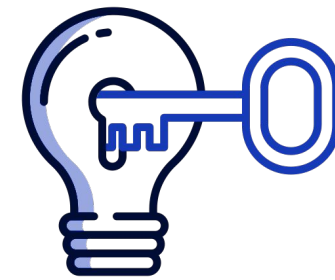
```
1  class SomeClass:
2      def __init__(self, value) -> None:
3          self._value = value
4
5      def __eq__(self, other):
6          return hash(self._value) == hash(other._value)
7
8      def __lt__(self, other):
9          return self._value < other._value
10
11  a = SomeClass(1)
12  b = SomeClass(2)
13
14  print(a < b)
15  # True
15  print(a.__lt__(b))
16  # True
17  print(a > b)
18  # False
19  print(a.__gt__(b))
20  # NotImplemented
```



Если класс определяет **изменяемый** тип объектов, то он не должен определять метод `__hash__`.

Декоратор total_ordering

```
1  from functools import total_ordering
2
3  @total_ordering
4  class SomeClass:
5      def __init__(self, value) -> None:
6          self._value = value
7
8      def __eq__(self, other):
9          return hash(self._value) == hash(other._value)
10
11     def __lt__(self, other):
12         return self._value < other._value
13
14  a = SomeClass(1)
15  b = SomeClass(2)
16
17  print(a > b)
18  # False
19  print(a.__gt__(b))
20  # False
```



Если в классе определены несколько методов расширенного сравнения, этот **декоратор** предоставит все остальные.

Это упрощает работу по определению всех возможных операций сравнения.

Арифметические операции

`__add__(self, other)`

`__sub__(self, other)`

`__mul__(self, other)`

`__matmul__(self, other)`

`__truediv__(self, other)`

`__floordiv__(self, other)`

`__mod__(self, other)`

`__divmod__(self, other)`

`__pow__(self, other[, modulo])`

Эти методы вызываются для реализации арифметических операций: **+**, **-**, *****, **@**, **/**, **//** и **%** соответственно.

Методы вызываются встроенными функциями **divmod()** и **pow()**. В последнюю можно передать 3 аргумента **base**, **exp** и **mod** (по умолчанию **None**) или вызвать оператором ****** с двумя аргументами.

Побитовые операции

`__lshift__(self, other)`

`__rshift__(self, other)`

`__and__(self, other)`

`__xor__(self, other)`

`__or__(self, other)`

Эти методы вызываются для реализации арифметических операций `<<`, `>>`, `&`, `^` и `|` соответственно.



Методы, соответствующие операциям, которые не поддерживаются конкретным типом реализуемых чисел (например, побитовые операции для нецелых чисел), следует оставить **неопределенными**.

Методы `__r*__` и `__i*__`

```
1 class SomeClass:
2     def __add__(self, other):
3         return type(self)(self._value + other._value)
4
5     def __iadd__(self, other):
6         print('Вызов метода __iadd__')
7         self._value = self._value + other._value
8         return self
9
10 class SomeOtherClass(SomeClass):
11     def __rsub__(self, other):
12         print('Вызов метода __rsub__')
13         return type(other)(other._value - self._value)
14
15 a, b = SomeClass(4), SomeOtherClass(2)
16 print(a)
17 # <__main__.SomeClass object at 0x107634fd0>
18 print(a + b)
19 # <__main__.SomeClass object at 0x107634d90>
20 a += b
21 # Вызов метода __iadd__
22 print(a)
23 # <__main__.SomeClass object at 0x107634fd0>
24 c = a - b
25 # Вызов метода __rsub__
26 print(c._value)
27 # 4
```

Методы `__i*__` должны стараться выполнить операцию изменив себя и вернуть результат который может быть самим собой.



Методы `__r*__` и `__i*__`

Отраженные операции	Расширенное присвоение
<code>__radd__(self, other)</code>	<code>__iadd__(self, other)</code>
<code>__rsub__(self, other)</code>	<code>__isub__(self, other)</code>
<code>__rmul__(self, other)</code>	<code>__imul__(self, other)</code>
<code>__rmatmul__(self, other)</code>	<code>__imatmul__(self, other)</code>
<code>__rtruediv__(self, other)</code>	<code>__itruediv__(self, other)</code>
<code>__rfloordiv__(self, other)</code>	<code>__ifloordiv__(self, other)</code>
<code>__rmod__(self, other)</code>	<code>__imod__(self, other)</code>
<code>__rdivmod__(self, other)</code>	
<code>__rpow__(self, other[, modulo])</code>	<code>__ipow__(self, other[, modulo])</code>

Отраженные операции	Расширенное присвоение
<code>__rlshift__(self, other)</code>	<code>__ilshift__(self, other)</code>
<code>__rrshift__(self, other)</code>	<code>__irshift__(self, other)</code>
<code>__rand__(self, other)</code>	<code>__iand__(self, other)</code>
<code>__rxor__(self, other)</code>	<code>__ixor__(self, other)</code>
<code>__ror__(self, other)</code>	<code>__ior__(self, other)</code>

Унарные операции

`__neg__(self)`

`__pos__(self)`

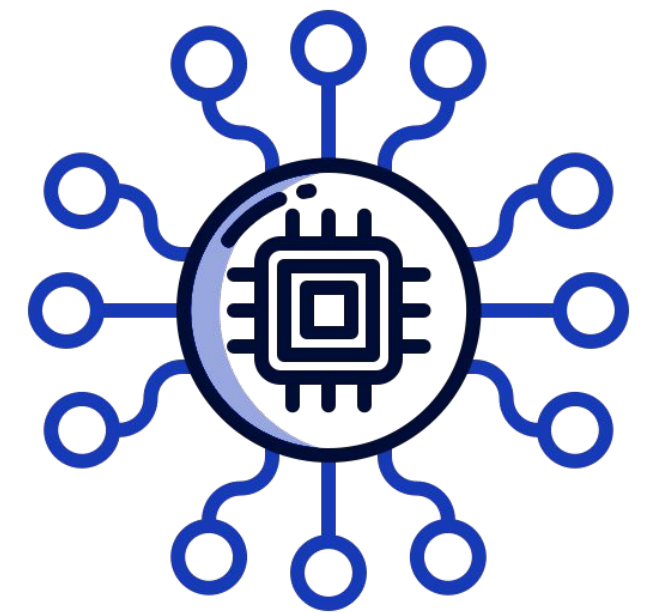
`__invert__(self)`

`__abs__(self)`

Методы вызываются для реализации унарных арифметических операций для `-`, `+`, `~` и `abs()` соответственно.

```
a = -10

print(-a)
# 10
print(+a)
# -10
print(~a)
# 9
print(abs(a))
# 10
```



Методы преобразования

`__complex__(self)`

`__int__(self)`

`__float__(self)`

`__index__(self)`

Методы вызываются для реализации встроенных функций `complex()`, `int()` и `float()` соответственно.

Используется когда Python нужно преобразовать числовой объект в целочисленный (например, при нарезке списков или во встроенных функциях `bin()`, `hex()`, `oct()`).



Если `__int__`, `__float__` и `__complex__` не определены, соответствующие встроенные функции `int()`, `float()` и `complex()` передают управление `__index__()`.

Методы преобразования

`__round__(self[, ndigits])`

`__trunc__(self)`

`__floor__(self)`

`__ceil__(self)`

Методы вызываются для реализации
встроенной функции `round()` и
математических функций `math.trunc()`,
`math.floor()` и `math.ceil()`.

```
import string

class Letter:
    def __init__(self, value):
        self._value = value

    def __index__(self):
        print('Вызов метода __index__')
        return self._value - 1

l1t = Letter(3)
alphabet = list(string.ascii_uppercase)
print(alphabet[l1t])
# Вызов метода __index__
# C
```

Методы представления

`__repr__(self)`

Методы вызываются встроенной функцией `repr()` для получения «официального» или «системного» представления объекта в виде строки.

Если это возможно метод должен возвращать допустимое выражение Python, в противном случае строку вида `< полезное описание >`.

`__str__(self)`

Методы вызываются встроенными функциями `format()` и `str()` для получения «неформального» представления объекта в виде строки.

```
import datetime

print(repr(object))
# <class 'object'>
now = datetime.datetime.now()
print(repr(now))
# datetime.datetime(2023, 3, 27, 14, 29, 4, 256282)
print(str(now))
# 2023-03-27 14:29:04.256282
```


Обзор dunder методов, часть 2



Доступ к атрибутам

`__getattr__(self, name)`

Метод вызывается всегда для доступа к атрибутам экземпляров класса. Чтобы избежать бесконечной рекурсии, при попытке доступа к атрибутам экземпляра, реализация должна вызывать одноименный метод базового класса.



Примечание: `__getattr__` не вызывается, если происходит поиск **dunder** метода, в результате вызова через синтаксис языка или встроенную функцию

```
class C:
    def __len__(self): return 10

    def __getattr__(*args):
        print('Вызов метода __getattr__')
        return object.__getattr__(*args)

c = C()
print(c.__len__())
# Вызов метода __getattr__
# 10
print(len(c))
# 10
```

Доступ к атрибутам

`__getattr__(self, name)`

Метод вызывается, когда доступ к атрибуту по умолчанию завершается ошибкой `AttributeError` или метод `__getattribute__` возвращает `AttributeError`.



Примечание: `__getattr__` не вызывается, если атрибут найден с помощью обычного механизма или с помощью `__getattribute__`

`__setattr__(self, name, value)`

Метод вызывается при попытке присвоения атрибута.

`__delattr__(self, name, value)`

Метод вызывается для удаления атрибута.

Вызываемые объекты

`__call__(self[, args...])`

Метод вызывается, когда экземпляр вызывается как функция.

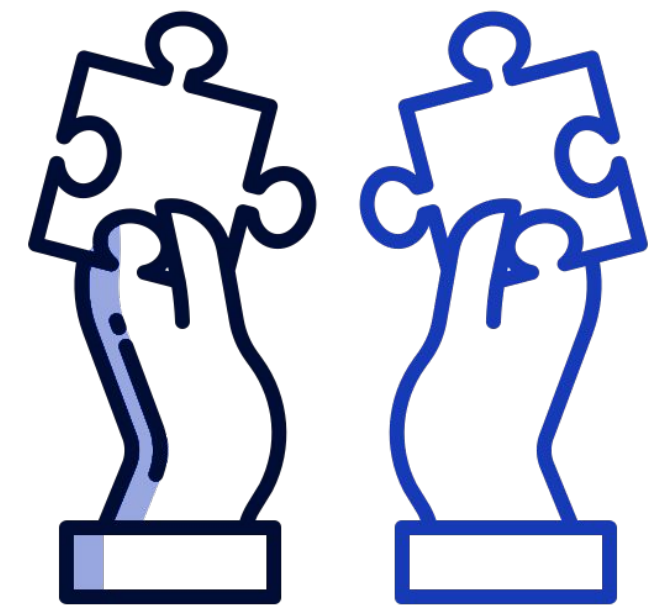
Если этот метод определен в классе, то `x(arg1, arg2, ...)`

исполняется примерно как `type(x).__call__(x, arg1, arg2, ...)`.

```
class RegexValidator:
    # some logic using __call__

integer_validator = RegexValidator(regex=r"^-\?\d+\Z")
slug_validator = RegexValidator(regex=r"^[-a-zA-Z0-9_]+\Z")

integer_validator('12345')
# True
slug_validator('\_(.*)_/')
# False
```



Менеджеры контекста

`__enter__(self)`

Вход в контекст среды исполнения, должен вернуть объект, связанный с контекстом среды выполнения. Обычная реализация менеджеров контекста включает в себя переключение состояния глобальных объектов, например блокировку и разблокировку ресурсов, открытие и закрытие файлов и т.д.

`__exit__(self, exc_type, exc_value, traceback)`

Выход из контекста среды исполнения, связанного с текущим файлом.

Аргументы `exc_type`, `exc_value` и `traceback` описывают исключение, вызвавшее выход из контекста. Если контекст завершился естественным путем, то все три аргумента имеют значение `None`.

Если менеджер контекста должен подавить исключение, то метод должен вернуть `True`, в противном случае исключение будет обработано обычным способом после выхода из контекста.

Классы дескрипторы

..... `__get__(self, instance, owner=None)`

Метод вызывается для получения доступа к атрибуту класса или атрибуту экземпляра. Метод должен возвращать вычисленное значение или `AttributeError`.

..... `__set__(self, instance, value)`

Метод вызывается, чтобы установить значение для атрибута **экземпляра** класса-владельца.

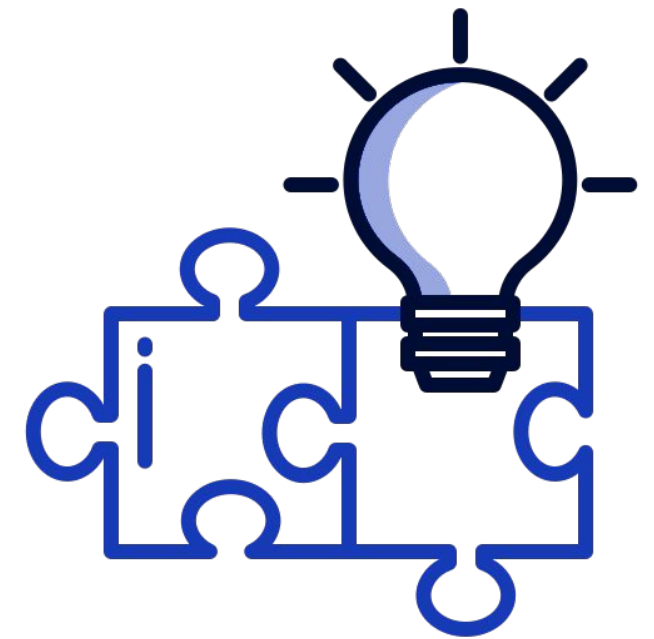
..... `__delete__(self, instance)`

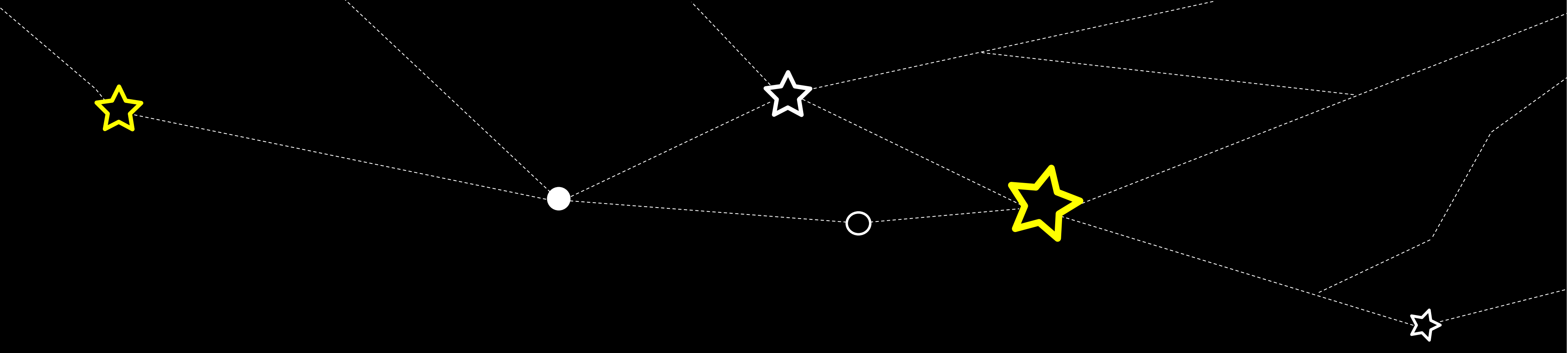
Метод вызывается, чтобы удалить значение атрибута.

Если класс определяет метод `__set__` или `__delete__` то такой дескриптор называется дескриптор данных, а если класс определяет только метод `__get__` то такой дескриптор будет называться дескриптор не данных (**non-data descriptor**).

Классы дескрипторы

```
1  class CounterDescriptor:
2      def __init__(self, value):
3          self._value = value
4          self._counter = 0
5
6      def __get__(self, instance, owner=None):
7          print('Вызов метода __get__')
8          self._counter += 1
9          return (self._value, self._counter)
10
11     def __set__(self, instance, value):
12         self._counter = 0
13         self._value = value
14
15 class ClassWithDescriptor:
16     x = CounterDescriptor(10)
17
18 f = ClassWithDescriptor()
19 f.x
20 # Вызов метода __get__
21 f.x
22 # Вызов метода __get__
23 print(f.x)
24 # Вызов метода __get__
25 # (10, 3)
26 f.x = 11
27 print(f.x)
28 # Вызов метода __get__
29 # (11, 1)
```





СПАСИБО ЗА ВНИМАНИЕ

Дмитрий Сергеев



@decembrin